

Overview of the SQL Query Language

The SQL language has several parts:

- **Data-definition language (DDL)**. The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
(Relation≈Table)

Relational Database Term	SQL/Practical Term
Relation	Table
Tuple	Row
Attribute	Column
Relation schema	Table structure/design

- **Data-manipulation language (DML)**. The SQL DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- **Integrity**. The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.

```
CREATE TABLE STUDENT (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(20),  
    Age INT CHECK (Age > 0)  
);
```

Example:

- **View definition**. The SQL DDL includes commands for defining views.

```
CREATE VIEW CSE_STUDENTS AS  
SELECT ID, Name  
FROM STUDENT  
WHERE Course = 'CSE';
```

Example:

- **Transaction control.** SQL includes [commands for specifying the beginning and ending of transactions.](#)

Example:

Suppose we want to transfer ₹500 from Account A to Account B.

```
</> SQL

BEGIN TRANSACTION;

UPDATE ACCOUNT
SET Balance = Balance - 500
WHERE ID = 101;

UPDATE ACCOUNT
SET Balance = Balance + 500
WHERE ID = 102;

COMMIT;
```

If something goes wrong, we can use:

```
</> SQL

ROLLBACK;
```

This cancels the changes made during the transaction.

Command	Meaning
BEGIN / START TRANSACTION	Start transaction
COMMIT	Save changes permanently
ROLLBACK	Undo changes

- **Embedded SQL and dynamic SQL.** Embedded and dynamic SQL define [how SQL statements can be embedded within general-purpose programming languages](#), such as C, C++, and Java.

Embedded = SQL written in the program.
Dynamic = SQL generated during program execution.

- **Authorization.** The SQL DDL includes [commands for specifying access rights to relations and views.](#)

Example:

The database administrator can give a user permission to **read** the table:

SQL

```
GRANT SELECT ON STUDENT TO Amit;
```

Now Amit can:

SQL

```
SELECT * FROM STUDENT;
```

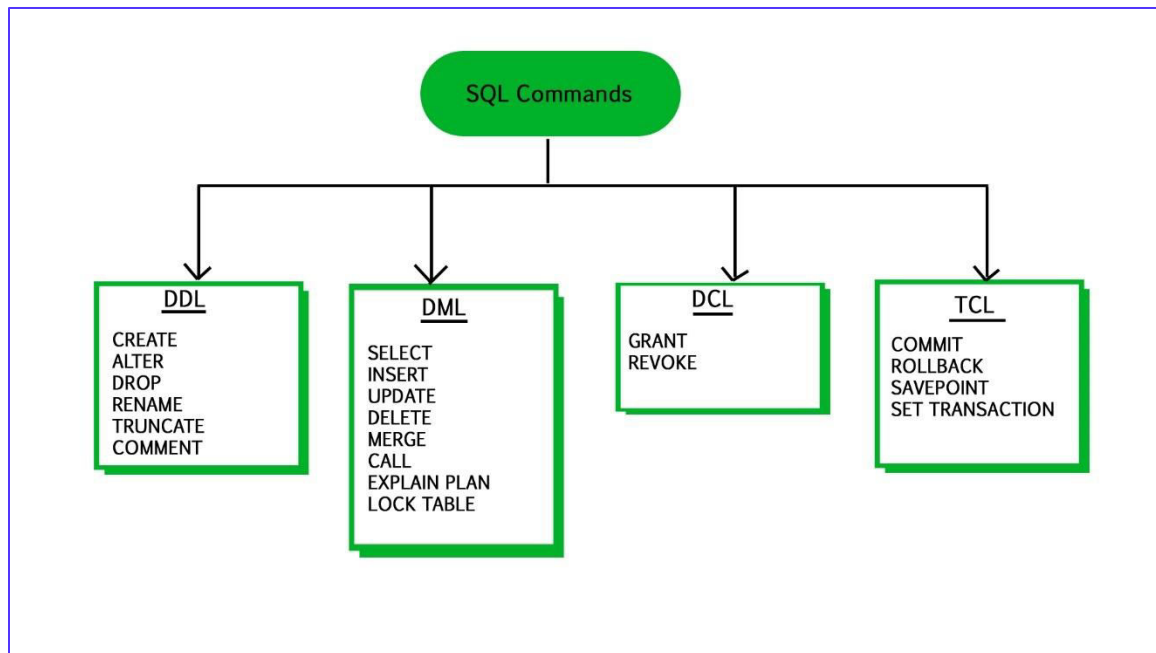
✓ Allowed — Amit can view the data.

But if Amit tries:

SQL

```
DELETE FROM STUDENT WHERE ID = 101;
```

✗ Not allowed — Amit has not been given `DELETE` permission.



- **DDL** → Data Definition Language
- **DML** → Data Manipulation Language
- **DCL** → Data Control Language
- **TCL** → Transaction Control Language

SQL Data Definition

The set of relations in a database must be specified to the system by means of a data-definition language (DDL).

The **SQL DDL allows** specification of not only a set of relations, but also information about each relation, including:

- The schema for each relation.
- The types of values associated with each attribute.
- The integrity constraints.
- The set of indices to be maintained for each relation. (**Indexing** is a data structure technique which allows you to quickly retrieve records from a database file. An Index is a small table having only two columns. The first column comprises a copy of the primary or

candidate key of a table. Its second column contains a set of **pointers** for holding the address of the disk block where that specific key value is stored.)

- The **security and authorization** information for each relation.
- The **physical storage structure of each relation** on disk.

If we search:

SQL

```
SELECT * FROM STUDENT WHERE ID = 103;
```

Without an index:

Check 101 → No
Check 102 → No
Check 103 → YES ✓

With an index:

Search ID 103
↓
INDEX
↓
Pointer
↓
Block 2
↓
Priya's record ✓

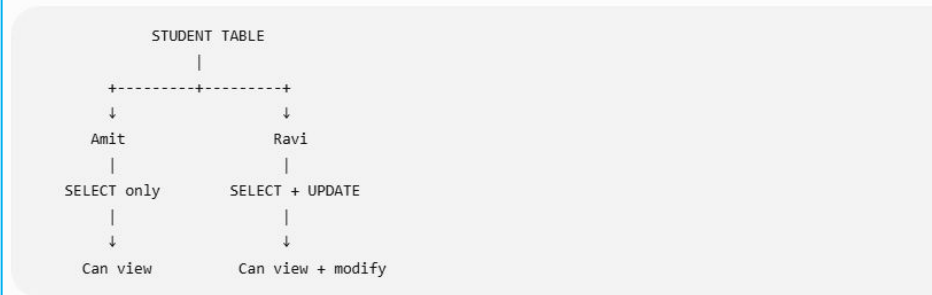
Example:

So, index = shortcut for finding data quickly.

Security and Authorization Information

This means the DBMS keeps information about **who can access a table and what they can do with it**.

For example:



For example:

```
</> SQL
GRANT SELECT ON STUDENT TO Amit;
```

Example: Amit can read the table.

3.2.1 Basic Types

The SQL standard supports a variety of built-in types, including:

- **char(*n*)**: A **fixed-length** **character string** with **user-specified length *n***. The full form, **character**, can be used instead.
- **varchar(*n*)**: A **variable-length** **character string** with **user-specified maximum length *n***. The full form, **character varying**, is equivalent.
- **int**: An integer (a finite subset of the integers that is machine dependent). The full form, **integer**, is equivalent.
- **smallint**: A small integer (a machine-dependent subset of the integer type).
- **numeric(*p*, *d*)**: A fixed-point number with user-specified precision. The number **consists of *p* digits** (plus a sign), and ***d* of the *p* digits are to the right of the decimal point**.
- **real, double precision**: Floating-point and double-precision **floating-point numbers with machine-dependent precision**.

- **float(*n*)**: A floating-point number, with precision of at least *n* digits.

Example:

Type	Simple meaning	Example
CHAR (<i>n</i>)	Fixed-length text	CHAR (10)
VARCHAR (<i>n</i>)	Variable-length text	VARCHAR (50)
INT	Whole number	25
SMALLINT	Small-range whole number	20
NUMERIC (<i>p</i> , <i>d</i>)	Exact decimal	NUMERIC (8, 2) → 123456.78
REAL	Floating-point number	3.14
DOUBLE PRECISION	More precise floating-point number	3.141592653
FLOAT (<i>n</i>)	Floating-point with at least <i>n</i> digits precision	FLOAT (10)

3.2.2 Basic Schema Definition

We define an SQL relation by using the **create table** command. The following command creates a relation *department* in the database.

```
create table department
(dept name varchar (20),    -- character string of maximum length 20
building varchar (15),     -- character string of maximum length 15
budget numeric (12,2),     -- which is a number with 12 digits in total, 2 of which are
                           after the decimal point
primary key (dept name));
```

The relation created above has three attributes, *dept name*, which is a character string of maximum length 20, *building*, which is a character string of maximum length 15, and *budget*, which is a number with 12 digits in total, 2 of which are

after the decimal point. The **create table** command also specifies that the *dept name* attribute is the primary key of the *department* relation.

A newly created relation is empty initially. We can use the **insert** command to load data into the relation.

For example, if we wish to insert the fact that there is an instructor named Smith in the Biology department with *instructor id* 10211 and a salary of \$66,000, we write:

```
insert into instructor values (10211, 'Smith', 'Biology', 66000);
```

We can use the **delete** command to delete tuples from a relation. The command

```
delete from student;
```

would delete all tuples from the *student* relation.

To remove a relation from an SQL database, we use the **drop table** command. The **drop table** command deletes all information about the dropped relation from the database. The command

```
drop table r;
```

is a more drastic action than

```
delete from r;
```

The latter retains relation *r*, but deletes all tuples in *r*. The former deletes not only all tuples of *r*, but also the schema for *r*. After *r* is dropped, no tuples can be inserted into *r* unless it is re-created with the **create table** command.

```
create table department
```

```
(dept name varchar (20),
```

building **varchar** (15),
budget **numeric** (12,2),
primary key (*dept name*));

create table *course*
(*course id* **varchar** (7),
title **varchar** (50),
dept name **varchar** (20),
credits **numeric** (2,0),
primary key (*course id*),
foreign key (*dept name*) **references** *department*);

create table *instructor*
(*ID* **varchar** (5),
name **varchar** (20) **not null**,
dept name **varchar** (20),
salary **numeric** (8,2),
primary key (*ID*),
foreign key (*dept name*) **references** *department*);

create table *section*
(*course id* **varchar** (8),
sec id **varchar** (8),
semester **varchar** (6),
year **numeric** (4,0),
building **varchar** (15),
room number **varchar** (7),
time slot id **varchar** (4),

primary key (*course id, sec id, semester, year*),

foreign key (*course id*) **references** *course*);

create table *teaches*

(*ID* **varchar** (5),

course id **varchar** (8),

sec id **varchar** (8),

semester **varchar** (6),

year **numeric** (4,0),

primary key (*ID, course id, sec id, semester, year*),

foreign key (*course id, sec id, semester, year*) **references** *section*,

foreign key (*ID*) **references** *instructor*);

[Figure 3.1 SQL data definition for part of the [university database](#).]

We use the **alter table** command to add attributes to an existing relation. All tuples in the relation are assigned *null* as the value for the new attribute. The form of the **alter table** command is

alter table *r* **add** *A* *D*;

SQL> **ALTER** TABLE salespeople **ADD** mobile varchar(10);

where *r* is the name of an existing relation, *A* is the name of the attribute to be added, and *D* is the type of the added attribute.

We can drop attributes from a relation by the command

alter table *r* **drop** *A*;

SQL> **ALTER** TABLE salespeople **DROP** COLUMN mobile;

where *r* is the name of an existing relation, and *A* is the name of an attribute of the

relation.

Many database systems do not support dropping of attributes, although they will allow an entire table to be dropped.

The **TRUNCATE TABLE** statement is used to delete the data inside a table, but not the table itself.

```
TRUNCATE TABLE table_name;
```

To rename a column in a table, use the following syntax:

```
ALTER TABLE table_name  
RENAME COLUMN old_name to new_name;
```

To change the data type of a column in a table, use the following syntax:

```
ALTER TABLE table_name  
MODIFY COLUMN column_name datatype;  
  
OR (After Oracle 10g)  
  
ALTER TABLE table_name  
MODIFY column_name datatype;
```

To create a **NOT NULL** constraint on the "Age" column when the "Persons" table is already created, use the following SQL: (will NOT accept NULL values)

```
ALTER TABLE Persons  
ALTER COLUMN Age int NOT NULL;
```

Basic Structure of SQL Queries

The basic structure of an SQL query consists of three clauses: **select**, **from**, and **where**. The query takes as its input the relations listed in the **from** clause, operates on them as specified in the **where** and **select** clauses, and then produces a relation as the result.

3.3.1 Queries on a Single Relation

“Find the names of all instructors.” Instructor names are found in the *instructor* relation put that relation in the **from** clause. The instructor’s name appears in the *name* attribute, so we put that in the **select** clause.

```
select name  
  
from instructor;
```

In practice, duplicate elimination is time-consuming. Therefore, SQL allows duplicates in relations as well as in the results of SQL expressions.

In those cases where we want to force the **elimination of duplicates**, we insert the keyword **distinct** after **select**. We can rewrite the preceding query as:

```
select distinct dept name  
  
from instructor;
```

SQL allows us to use the keyword **all** to specify **explicitly** that duplicates are not removed:

```
select all dept name  
  
from instructor;
```

Since duplicate retention is the **default**, we shall not use **all** in our examples.

To ensure the elimination of duplicates in the results of our example queries, we shall use **distinct** whenever it is necessary.

The **select** clause may also contain arithmetic expressions involving the operators +, −, *, and / operating on constants or attributes of tuples. For example, the query:

```
select ID, name, dept name, salary * 1.1  
from instructor;
```

The **where** clause allows us to select only those rows in the result relation of the **from** clause that satisfy a specified predicate.

Consider the query “Find the names of all instructors in the Computer Science department who have salary greater than \$70,000.” This query can be written in SQL as:

```
select name  
from instructor  
where dept name = 'Comp. Sci.' and salary > 70000;
```

SQL allows the use of the logical connectives **and, or, and not in the where clause**. The operands of the logical connectives can be expressions involving the **comparison operators <, <=, >, >=, =, and <>**. SQL allows us to use the comparison operators to compare strings and arithmetic expressions, as well as special types, such as date types.

3.3.2 Queries on Multiple Relations

Queries often need to access information from multiple relations.

An example, suppose we want to answer the query “Retrieve the names of all instructors, along with their department names and department building name.”

Looking at the schema of the relation *instructor*, we realize that we can get the department name from the attribute *dept name*, but the department building name is present in the attribute *building* of the relation *department*.

```
select name, instructor.dept name, building
from instructor, department
where instructor.dept name= department.dept name;
```

The role of each clause is as follows:

- The **select** clause is **used to list the attributes** desired in the result of a query.
- The **from** clause is a **list of the relations to be accessed** in the evaluation of the query.
- The **where** clause is a **predicate involving attributes of the relation in the from clause**.

We would expect a query involving *instructor* and *teaches* to combine a particular tuple *t* in *instructor* with only those tuples in *teaches* that refer to the same instructor to which *t* refers. That is, we wish only to **match *teaches* tuples with *instructor* tuples** that have the same *ID* value. The following SQL query ensures this condition, and outputs the instructor name and course identifiers from such

matching tuples.

```
select name, course id
from instructor, teaches
where instructor.ID= teaches.ID;
```

If we only wished to find instructor names and course identifiers for instructors in the Computer Science department, we could add an extra predicate to the **where** clause, as shown below.

```
select name, course id
from instructor, teaches
where instructor.ID= teaches.ID and instructor.dept name = 'Comp. Sci.';
```

3.3.3 The Natural Join

Consider the query “For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught”, which we wrote earlier as:

```
select name, course id
from instructor, teaches
where instructor.ID= teaches.ID;
```

This query can be written more concisely using the natural-join operation in SQL as:

```
select name, course id
from instructor natural join teaches;
```

Both of the above queries generate the same result.

For example, suppose we wish to answer the query “List the names of instructors along with the titles of courses that they teach.” The query can be written in SQL as follows:

```
select name, title
from instructor natural join teaches, course
where teaches.course id= course.course id;
```

The natural join of *instructor* and *teaches* is first computed, as we saw earlier, and a Cartesian product of this result with *course* is computed, from which the **where** clause extracts only those tuples where the course identifier from the join result matches the course identifier from the *course* relation. Note that *teaches.course id* in the **where** clause refers to the *course id* field of the natural join result, since this field in turn came from the *teaches* relation.

In contrast the following SQL query does *not* compute the same result:

```
select name, title
from instructor natural join teaches natural join course;
```

To see why, note that the natural join of *instructor* and *teaches* contains the attributes (*ID*, *name*, *dept name*, *salary*, *course id*, *sec id*), while the *course* relation contains the attributes (*course id*, *title*, *dept name*, *credits*). As a result, the natural join of these two would require that the *dept name* attribute values from the two inputs be the same, in addition to requiring that the *course id* values be the same. This query would then omit all (instructor name, course title) pairs where the instructor teaches a course in a department other than the instructor's own department.

3.4 Additional Basic Operations

There are number of additional basic operations that are supported in SQL.

3.4.1 The Rename Operation

Consider again the query that we used earlier:

```
select name, course id
from instructor, teaches
where instructor.ID= teaches.ID;
```

The result of this query is a relation with the following attributes:

name, course id

The names of the attributes in the result are derived from the names of the attributes in the relations in the **from** clause.

We cannot, however, always derive names in this way, for several reasons:

First, two relations in the **from** clause may have attributes with the same name, in which case an attribute name is duplicated in the result. **Second**, if we used an arithmetic expression in the **select**

clause, the resultant attribute does not have a name. **Third**, even if an attribute name can be derived from the base relations as in the preceding example, we may want to change the attribute name in the result. **Hence**, SQL provides a way of renaming the attributes of a result relation.

It uses the **as clause**, taking the form:

old-name **as** *new-name*

The **as** clause can appear in both the **select** and **from** clauses.

For example, if we want the attribute name *name* to be replaced with the name *instructor name*, we can rewrite the preceding query as:

```
select name as instructor name, course id
from instructor, teaches
where instructor.ID= teaches.ID;
```

The **as** clause is particularly useful in renaming relations. **One reason** to rename a relation is to replace a long relation name with a shortened version that is more convenient to use elsewhere in the query. To illustrate, we rewrite the query “For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught.”

```
select T.name, S.course id
from instructor as T, teaches as S
where T.ID= S.ID;
```

Another reason to rename a relation is a case where we wish to compare tuples in the same relation. Suppose that we want to write the query “Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.” We can write the SQL expression:

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept name = 'Biology';
```

Observe that we could not use the notation *instructor.salary*, since it would not be clear which reference to *instructor* is intended.

In the above query, *T* and *S* can be thought of as copies of the relation *instructor*, **but** more precisely, they are declared as aliases, that is as alternative names, for the relation *instructor*. An identifier, such as *T* and *S*, that is used to rename a relation is referred to as a **correlation name** in the SQL standard, but is also commonly referred to as a **table alias**, or a **correlation variable**, or a **tuple variable**.

3.4.2 String Operations

SQL specifies strings by enclosing them in single quotes, for example, 'Computer'. A single quote character that is part of a string can be specified by using two single quote characters; for example, the string "It's right" can be specified by "It's right".

The SQL standard specifies that the equality operation on strings is case sensitive; as a result the expression "'comp. sci.' = 'Comp. Sci.'" evaluates to false.

However, some database systems, such as MySQL and SQL Server, do not distinguish uppercase from lowercase when matching strings; as a result "'comp. sci.' = 'Comp. Sci.'" would evaluate to true on these databases. This default behavior can, however, be changed, either at the database level or at the level of specific attributes.

SQL also permits a variety of functions on character strings, such as concatenating (using "||"), extracting substrings, finding the length of strings, converting strings to uppercase (using the function upper(s) where s is a string) and lowercase (using the function lower(s)), removing spaces at the end of the string (using trim(s)) and so on. There are variations on the exact set of string functions supported by different database systems. See your database system's manual for more details on exactly what string functions it supports.

Pattern matching can be performed on strings, using the operator like. We describe patterns by using two special characters:

- Percent (%): The % character matches any substring.
- Underscore (_): The character matches any character.

Patterns are case sensitive; that is, uppercase characters do not match lowercase characters, or vice versa. To illustrate pattern matching, we consider the following examples:

- 'Intro%' matches any string beginning with "Intro".
- '%Comp%' matches any string containing "Comp" as a substring, for example,

'Intro. to Computer Science', and 'Computational Biology'.

- '___' matches any string of exactly three characters.
- '___%' matches any string of at least three characters.

SQL expresses patterns by using the like comparison operator. Consider the query "Find the names of all departments whose building name includes the substring 'Watson'." This query can be written as:

```
select dept name  
  
from department  
  
where building like '%Watson%';
```

For patterns to include the special pattern characters (that is, % and _), SQL allows the specification of an escape character. The **escape character** is used immediately before a special pattern character to indicate that the special pattern character is to be treated like a normal character. We define the escape character for a **like** comparison using the **escape** keyword. To illustrate, consider the following patterns, which use a backslash (\) as the escape character:

- like 'ab\%cd%' escape '\ ' matches all strings beginning with "ab%cd".
- like 'ab\cd%' escape '\ ' matches all strings beginning with "ab\cd".

SQL allows us to search for mismatches instead of matches by using the **not like** comparison operator. Some databases provide variants of the **like** operation which do not distinguish lower and upper case.

3.4.3 Attribute Specification in Select Clause

The asterisk symbol "*" can be used in the **select** clause to denote "all attributes."

Thus, the use of *instructor.** in the **select** clause of the query:

```
select instructor.*  
  
from instructor, teaches  
  
where instructor.ID= teaches.ID;
```

indicates that all attributes of *instructor* are to be selected. A **select** clause of the form select * indicates that all attributes of the result relation of the **from** clause are selected.

3.4.4 Ordering the Display of Tuples

SQL offers the user some control over the order in which tuples in a relation are displayed. The order by clause causes the tuples in the result of a query to appear in sorted order. To list in alphabetic order all instructors in the Physics department, we write:

```
select name  
  
from instructor  
  
where dept name = 'Physics'
```

```
order by name;
```

By default, the **order by** clause lists items in ascending order. To specify the sort order, we may specify **desc** for descending order or **asc** for ascending order.

Furthermore, ordering can be performed on multiple attributes. “Suppose that we wish to list the entire *instructor* relation in descending order of *salary*. If several instructors have the same salary, we order them in ascending order by name”. We express this query in SQL as follows:

```
select *  
  
from instructor  
  
order by salary desc, name asc;
```

3.4.5 Where Clause Predicates

SQL includes a **between comparison operator** to simplify **where clauses** that specify that a value be less than or equal to some value and greater than or equal to some other value. If we wish to find the names of instructors with salary amounts between \$90,000 and \$100,000, we can use the **between** comparison to write:

```
select name  
  
from instructor  
  
where salary between 90000 and 100000;
```

instead of:

```
select name  
  
from instructor  
  
where salary <= 100000 and salary >= 90000;
```

Similarly, we can use the **not between comparison operator**. We can extend the preceding query that finds instructor names along with course identifiers, which we saw earlier, and consider a more complicated case in which we require also that the instructors be from the Biology department:

“Find the instructor names and the courses they taught for all instructors in the Biology department who have taught some course.” To write this query, we can modify either of the SQL queries we saw earlier, by adding an extra condition in the **where** clause. We show below the modified form of the SQL query that does not use natural join.

```
select name, course id
from instructor, teaches
where instructor.ID= teaches.ID and dept name = 'Biology';
```

The preceding SQL query can be rewritten as follows:

```
select name, course id
from instructor, teaches
where (instructor.ID, dept name) = (teaches.ID, 'Biology');
```

3.5 Set Operations

The SQL operations **union**, **intersect**, and **except** operate on relations and correspond to the mathematical set-theory operations $\cup$, $\cap$, and $-$. We shall now construct queries involving the **union**, **intersect**, and **except** operations over two sets.

- The set of all courses taught in the Fall 2009 semester:

```
select course id
from section
where semester = 'Fall' and year= 2009;
```

- The set of all courses taught in the Spring 2010 semester:

```
select course id
from section
where semester = 'Spring' and year= 2010;
```

3.5.1 The Union Operation

To find the set of all courses taught either in Fall 2009 or in Spring 2010, or both,

we write:

```
(select course id
from section
where semester = 'Fall' and year= 2009)
```

union

```
(select course id  
  
from section  
  
where semester = 'Spring' and year= 2010);
```

The **union** operation automatically **eliminates duplicates**, unlike the **select clause**. Thus, using the **If** we want to retain all duplicates, we must write **union all** in place of **union**:

```
(select course id  
  
from section  
  
where semester = 'Fall' and year= 2009)
```

union all

```
(select course id  
  
from section  
  
where semester = 'Spring' and year= 2010);
```

Note: The parentheses we include around each **select-from-where** statement are optional, but useful for ease of reading.

3.5.2 The Intersect Operation

To find the set of all courses taught in the Fall 2009 **as well as** in Spring 2010 we write:

```
(select course id  
  
from section  
  
where semester = 'Fall' and year= 2009)
```

intersect

```
(select course id  
  
from section  
  
where semester = 'Spring' and year= 2010);
```

If we want to retain all duplicates, we must write **intersect all** in place of **intersect**:

```
(select course id  
from section  
where semester = 'Fall' and year= 2009)
```

intersect all

```
(select course id  
from section where semester = 'Spring' and year= 2010);
```

3.5.3 The Except Operation

To find **all** courses taught in the Fall 2009 semester **but not** in the Spring 2010 semester, we write:

```
(select course id  
from section  
where semester = 'Fall' and year= 2009)
```

except

```
(select course id  
from section  
where semester = 'Spring' and year= 2010);
```

The operation automatically eliminates duplicates in the inputs before performing set difference. For example, If we want to retain duplicates, we must write **except all** in place of **except**:

```
(select course id  
from section  
where semester = 'Fall' and year= 2009)
```

except all

```
(select course id  
from section
```

where *semester* = 'Spring' **and** *year* = 2010);

3.6 Null Values

Null values present special problems in relational operations, including arithmetic operations, comparison operations, and set operations.

Since the predicate in a **where** clause can involve Boolean operations such as **and**, **or**, and **not** on the results of comparisons, the definitions of the Boolean operations are extended to deal with the value **unknown**.

- **and**: The result of true **and** unknown is unknown, false **and** unknown is false,

while unknown **and** unknown is unknown.

- **or**: The result of true **or** unknown is true, false **or** unknown is unknown, while

unknown **or** unknown is unknown.

- **not**: The result of **not** unknown is unknown.

If the **where** clause predicate evaluates to either **false** or **unknown** for a tuple, that tuple is not added to the result.

SQL uses the special keyword **null** in a predicate to test for a null value. Thus, to find all instructors who appear in the *instructor* relation with null values for *salary*, we write:

```
select name
from instructor
where salary is null;
```

3.7 Aggregate Functions

Aggregate functions are functions that take a collection (a set or multiset) of values as input and return a single value. SQL offers five built-in aggregate functions:

- Average: **avg**

- Minimum: **min**
- Maximum: **max**
- Total: **sum**
- Count: **count**

The input to **sum** and **avg** must be a collection of numbers, **but** the other operators can operate on collections of non-numeric data types, such as strings, as well.

3.7.1 Basic Aggregation

Consider the query “Find the average salary of instructors in the Computer Science department.” We write this query as follows:

```
select avg (salary)
from instructor
where dept name= 'Comp. Sci.';
```

The result of this query is a relation with a single attribute, containing a single tuple with a numerical value corresponding to the average salary of instructors in the Computer Science department. The database system may give an absolute name to the result relation attribute that is generated by aggregation; however, we can give a meaningful name to the attribute by using the as clause as follows:

```
select avg (salary) as avg salary
from instructor
where dept name= 'Comp. Sci.';
```

There are cases where we must eliminate duplicates before computing an aggregate function. If we do want to eliminate duplicates, we use the keyword **distinct** in the aggregate expression. An example arises in the query “Find the total number of instructors who teach a course in the Spring 2010 semester.”

In this case, an instructor counts only once, regardless of the number of course sections that the instructor teaches. The required information is contained in the relation *teaches*, and we write this query as follows:

```
select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2010;
```

Because of the keyword **distinct** preceding *ID*, even if an instructor teaches more than one course, *ID* is counted only once in the result.

We use the aggregate function **count** frequently to count the number of tuples in a relation. The notation for this function in SQL is **count (*)**. Thus, to find the number of tuples in the *course* relation, we write

```
select count (*)  
from course;
```

SQL does not allow the use of **distinct** with **count (*)**. It is legal to use **distinct** with **max** and **min**, even though the result does not change. We can use the keyword **all** in place of **distinct** to specify duplicate retention, but since **all** is the default, there is no need to do so.

3.7.2 Aggregation with Grouping

There are circumstances where we would like to apply the aggregate function not only to a single set of tuples, but also to a group of sets of tuples; we specify this wish in SQL using the **group by** clause. The attribute or attributes given in the **group by** clause are used to form groups. Tuples with the same value on all attributes in the **group by** clause are placed in one group.

As an illustration, consider the query “Find the average salary in each department.”

We write this query as follows:

```
select dept name, avg (salary) as avg salary  
from instructor  
group by dept name;
```

In contrast, consider the query “Find the average salary of all instructors.” We write this query as follows:

```
select avg (salary)  
from instructor;
```

3.7 Aggregate Functions

As another example of aggregation on groups of tuples, consider the query “Find the number of instructors in each department who teach a course in the Spring 2010 semester.” Information about which instructors teach which course sections in which semester is available in the *teaches* relation. However, this information has to be joined with information from the *instructor* relation to get the department name of each instructor. Thus, we write this query as follows:

```
select dept name, count (distinct ID) as instr count
from instructor natural join teaches
where semester = 'Spring' and year = 2010
group by dept name;
```

The result relation for the query “Find the number of instructors in each department who teach a course in the Spring 2010 semester.”

/ erroneous query */*

```
select dept name, ID, avg (salary)
from instructor
group by dept name;
```

Each instructor in a particular group (defined by *dept name*) can have a different *ID*, and since only one tuple is output for each group, there is no unique way of choosing which *ID* value to output. As a result, such cases are disallowed by SQL.

3.7.3 The Having Clause

At times, it is useful to state a condition that applies to groups rather than to tuples. For example, we might be interested in only those departments where the average salary of the instructors is more than \$42,000. This condition does not apply to a single tuple; rather, it applies to each group constructed by the **group by** clause. To express such a query, we use the **having** clause of SQL. SQL applies predicates in the **having** clause after groups have been formed, so aggregate functions may be used. We express this query in SQL as follows:

```
select dept name, avg (salary) as avg salary
from instructor
group by dept name
having avg (salary) > 42000;
```

The meaning of a query containing aggregation, **group by**, or **having** clauses is defined by the following sequence of operations:

1. As was the case for queries without aggregation, the **from** clause is first evaluated to get a relation.

dept name avg(avg salary)

2. If a **where** clause is present, the predicate in the **where** clause is applied on the result relation of the **from** clause.

3. Tuples satisfying the **where** predicate are then placed into groups by the **group by** clause if it is present. If the **group by** clause is absent, the entire set of tuples satisfying the **where** predicate is treated as being in one group.

4. The **having** clause, if it is present, is applied to each group; the groups that do not satisfy the **having** clause predicate are removed.

5. The **select** clause uses the remaining groups to generate tuples of the result of the query, applying the aggregate functions to get a single result tuple for each group.

To illustrate the use of both a **having** clause and a **where** clause in the same query, we consider the query “For each **course section** offered in 2009, find the average total credits (*tot cred*) of all students enrolled in the **section**, if the section had at least 2 students.”

```
select course id, semester, year, sec id, avg (tot cred)
from takes natural join student
where year = 2009
group by course id, semester, year, sec id
having count (ID) >= 2;
```

Note that all the required information for the preceding query is available from the relations *takes* and *student*.

3.7.4 Aggregation with **Null** and **Boolean** Values

Null values, when they exist, complicate the processing of aggregate operators. For example, assume that some tuples in the *instructor* relation have a **null** value for *salary*. Consider the following query to total all salary amounts:

```
select sum (salary)
from instructor;
```

The values to be summed in the preceding query include null values, since some tuples have a null value for *salary*. Rather than say that the overall sum is itself *null*, the **SQL standard** says that the **sum** operator should ignore *null* values in its input.

In general, aggregate functions treat nulls according to the following rule: All aggregate functions except **count (*)** ignore null values in their input collection. As a result of null values being ignored, the collection of values may be empty. The **count** of an empty collection is defined to be 0, and all other aggregate operations return a value of null when applied on an empty collection. The effect of null values on some of the more complicated SQL constructs can be harmful.

A **Boolean** data type that can take values **true**, **false**, and **unknown**. The aggregate functions **some** and **every**, which mean exactly what you would intuitively expect, can be applied on a collection of Boolean values.

3.8 Nested Subqueries

SQL provides a mechanism for **nesting subqueries**. A subquery is a **select-from where** expression that is **nested within another query**. A common use of subqueries is to perform tests for set membership, make set comparisons, and determine set cardinality, by **nesting subqueries in the where clause**.

3.8.1 Set Membership

SQL **allows testing tuples for membership in a relation**. The **in** connective tests for set membership, where the **set** is a **collection of values produced by a select clause**. The **not in** connective tests for the absence of set membership.

As an illustration, reconsider the query “Find all the courses taught in the both the Fall 2009 and Spring 2010 semesters.” Earlier, we wrote such a query by **intersecting two sets**: the set of courses taught in Fall 2009 and the set of courses taught in Spring 2010. We can take the **alternative approach of finding all courses that were taught in Fall 2009 and that are also members of the set of courses taught in Spring 2010**. Clearly, this formulation generates the same results as the previous one did, but it leads us to write our query using the **in** connective of SQL. We **begin by finding all courses taught in Spring 2010**, and we write the subquery

```
(select course id
from section
where semester = 'Spring' and year= 2010)
```

We **then** need to find those courses that were taught in the Fall 2009 and that appear in the set of courses obtained in the subquery. We do so by nesting the subquery in the **where** clause of an outer query. The resulting query is

```
select distinct course id
from section
where semester = 'Fall' and year= 2009 and
       course id in (select course id
                     from section
                     where semester = 'Spring' and year= 2010);
```

We use the **not in** construct in a way similar to the **in** construct. For example, to find all the courses taught in the Fall 2009 semester but not in the Spring 2010 semester, we can write:

```
select distinct course id
from section
where semester = 'Fall' and year= 2009 and
       course id not in (select course id
                        from section
                        where semester = 'Spring' and year= 2010);
```

The **in** and **not in** operators can also be used on enumerated sets. The following query selects the names of instructors whose names are neither “Mozart” nor “Einstein”.

```
select distinct name
from instructor
where name not in ('Mozart', 'Einstein');
```

In the preceding examples, we tested membership in a one-attribute relation. It is also possible to test for membership in an arbitrary relation in SQL. For example, we can write the query “find the total number of (distinct) students who have taken course sections taught by the instructor with ID 110011” as follows:

```
select count (distinct ID)
from takes
where (course id, sec id, semester, year) in (select course id, sec id, semester, year
```

```
from teaches
where teaches.ID= 10101);
```

3.8.2 Set Comparison

As an example of the ability of a [nested subquery to compare sets](#), consider the query “Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.” In Section 3.4.1, we wrote this query as follows:

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept name = 'Biology';
```

SQL does, however, offer an alternative style for writing the preceding query. The phrase “[greater than at least one](#)” is represented in SQL by [> some](#). This construct allows us to rewrite the query in a form that resembles closely our formulation of the query in English.

```
select name
from instructor
where salary > some (select salary
                      from instructor
                      where dept name = 'Biology');
```

The [subquery](#):

```
(select salary
 from instructor
 where dept name = 'Biology')
```

generates the set of all salary values of all instructors in the Biology department. The [> some](#) comparison in the **where** clause of the outer **select** is true if the *salary* value of the tuple is greater than at least one member of the set of all salary values for instructors in Biology.

SQL also allows **< some**, **<= some**, **>= some**, **= some**, and **<> some** comparisons. As an exercise, verify that **= some** is identical to **in**, whereas **<> some** is not the same as **not in**.

Now we modify our query slightly. Let us find the names of all instructors that have a salary value greater than that of **each** instructor in the Biology department. The construct **> all** corresponds to the phrase “greater than all.” Using this construct, we write the query as follows:

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept name = 'Biology');
```

As it does for **some**, SQL also allows **< all**, **<= all**, **>= all**, **= all**, and **<> all** comparisons. As an exercise, verify that **<> all** is identical to **not in**, whereas **= all** is not the same as **in**.

As another example of set comparisons, consider the query “Find the departments that have the highest average salary.” We begin by writing a query to find all average salaries, and then nest it as a subquery of a larger query that finds those departments for which the average salary is greater than or equal to all

average salaries:

```
select dept name
from instructor
group by dept name
having avg (salary) >= all (select avg (salary)
                          from instructor
                          group by dept name);
```

3.8.3 Test for Empty Relations

SQL includes a feature for testing whether a subquery has any tuples in its result. The **exists** construct returns the value **true** if the argument subquery is **nonempty**. Using the **exists** construct, we can write

the query “Find all courses taught in **both** the Fall 2009 semester and in the Spring 2010 semester” in still another way:

```
select course id
from section as S
where semester = 'Fall' and year= 2009 and
    exists (select *
            from section as T
            where semester = 'Spring' and year= 2010 and
                S.course id= T.course id);
```

The above query also illustrates a feature of SQL where a correlation name from an outer query (S in the above query), can be used in a subquery in the **where** clause. A subquery that uses a correlation name from an outer query is called a **correlated subquery**.

In queries that contain subqueries, a **scoping rule applies for correlation names**. In a subquery, according to the rule, it is legal to use only correlation names defined in the subquery itself or in any query that contains the subquery.

If a correlation name is defined both locally in a subquery and globally in a containing query, the local definition applies. This rule is analogous to the usual scoping rules used for variables in programming languages. We can **test for the nonexistence of tuples in a subquery** by using the **not exists** construct. We can use the **not exists** construct to simulate the set containment (that is, superset) operation: We can write “relation *A* contains relation *B*” as “**not exists (*B* except *A*)**.” (Although it is not part of the current SQL standards, the **contains** operator was present in some early relational systems.) To illustrate the **not exists** operator, consider the query “Find all students who have taken all courses offered in the Biology department.” Using the **except** construct, we can write the query

as follows:

```
select distinct S.ID, S.name
from student as S
where not exists ((select course id
                  from course
                  where dept name = 'Biology')
```

```

except

(select T.course id

from takes as T

where S.ID = T.ID));

```

Here, the subquery:

```

(select course id

from course

where dept name = 'Biology')

```

finds the set of all courses offered in the Biology department. The subquery:

```

(select T.course id

from takes as T

where S.ID = T.ID)

```

finds all the courses that student *S.ID* has taken.

Thus, the outer **select** takes each student and tests whether the set of all courses that the student has taken contains the set of all courses offered in the Biology department.

3.8.4 Test for the Absence of Duplicate Tuples

SQL includes a boolean function for testing whether a subquery has duplicate tuples in its result. The **unique** construct returns the value **true** if the argument subquery contains **no duplicate tuples**. Using the **unique** construct, we can write the query “Find all courses that were offered at most once in 2009” as follows:

```

select T.course id
from course as T
where unique (select R.course id
               from section as R
               where T.course id = R.course id and
                  R.year = 2009);

```

Note that if a course is not offered in 2009, the subquery would return an empty result, and the **unique** predicate would evaluate to true on the empty set.

An equivalent version of the above query not using the **unique** construct is:

```
select T.course id
from course as T
where 1 <= (select count(R.course id)
            from section as R
            where T.course id= R.course id and
            R.year = 2009);
```

We can test for the existence of duplicate tuples in a subquery by using the **not unique** construct. To illustrate this construct, consider the query “Find all courses that were offered at least twice in 2009” as follows:

```
select T.course id
from course as T
where not unique (select R.course id
                  from section as R
                  where T.course id= R.course id and
                  R.year = 2009);
```

3.8.5 Subqueries in the From Clause

SQL allows a subquery expression to be used in the **from** clause. The key concept applied here is that any **select-from-where** expression returns a relation as a result and, therefore, can be inserted into another **select-from-where** anywhere that a relation can appear. Consider the query “Find the average instructors’ salaries of those departments where the average salary is greater than \$42,000.” We wrote this query in Section 3.7 by using the **having** clause. We can now rewrite this query, without using the **having** clause, by using a subquery in the **from** clause, as follows:

```
select dept name, avg salary
from (select dept name, avg (salary) as avg salary
      from instructor
      group by dept name)
```

where *avg salary* > 42000;

We note that nested subqueries in the **from** clause cannot use correlation variables from other relations in the **from** clause. However, SQL:2003 allows a subquery in the **from** clause that is prefixed by the **lateral** keyword to access attributes of preceding tables or subqueries in the **from** clause. For example, if we wish to print the names of each instructor, along with their salary and the average salary in their department, we could write the query as follows:

```
select name, salary, avg salary
from instructor I1, lateral (select avg(salary) as avg salary
                               from instructor I2
                               where I2.dept name= I1.dept name);
```

Without the lateral clause, the subquery cannot access the correlation variable *I1* from the outer query. Currently.

3.8.6 The **with** Clause

The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs. Consider the following query, which **finds those departments with the maximum budget**.

```
with max budget (value) as
    (select max(budget)
     from department)
select budget
from department, max budget
where department.budget = max budget.value;
```

The **with** clause defines the temporary relation *max budget*, which is used in the immediately following query.

3.8.7 Scalar Subqueries

SQL allows subqueries to occur wherever an expression returning a value is permitted, **provided the subquery returns only one tuple containing a single attribute**; such subqueries are called **scalar subqueries**. For example, a subquery.

can be used in the **select** clause as illustrated in the following example that lists all departments along with the number of instructors in each department:

```
select dept name,  
  
       (select count(*)  
  
        from instructor  
  
        where department.dept name = instructor.dept name)  
  
       as num instructors  
  
from department;
```

The subquery in the above example is guaranteed to return only a single value since it has a **count(*)** aggregate without a **group by**.

Scalar subqueries can occur in **select**, **where**, and **having** clauses.

3.9.1 Deletion

```
delete from instructor;
```

deletes all tuples from the *instructor* relation. The *instructor* relation itself still exists, but it is empty.

Here are examples of SQL delete requests:

- Delete all tuples in the *instructor* relation pertaining to instructors in the

Finance department.

```
delete from instructor  
  
where dept name= 'Finance';
```

- Delete all instructors with a salary between \$13,000 and \$15,000.

```
delete from instructor  
  
where salary between 13000 and 15000;
```

- Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building.

```
delete from instructor  
  
where dept name in (select dept name  
  
from department  
  
where building = 'Watson');
```

This **delete** request first finds all departments located in Watson, and then deletes all *instructor* tuples pertaining to those departments.

Note that, although we may delete tuples from only one relation at a time, we may reference any number of relations in a **select-from-where** nested in the **where** clause of a **delete**. The **delete** request can contain a nested **select** that references the relation from which tuples are to be deleted. For example, suppose that we want to delete the records of all instructors with salary below the average at the university. We could write:

```
delete from instructor  
  
where salary < (select avg (salary)  
  
from instructor);
```

3.9.2 Insertion

```
insert into course  
  
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

insert into course (course id, title, dept name, credits)

```
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);  
  
insert into course (title, course id, credits, dept name)  
  
values ('Database Systems', 'CS-437', 4, 'Comp. Sci.');
```

```
insert into instructor  
  
select ID, name, dept name, 18000  
  
from student  
  
where dept name = 'Music' and tot cred > 144;
```

Instead of specifying a tuple as we did earlier in this section, we use a **select** to specify a set of tuples. SQL evaluates the **select** statement first, giving a set of tuples that is then inserted into the *instructor* relation.

3.9.3 Updates

```
update instructor  
  
set salary = salary * 1.05;
```

```
update instructor  
  
set salary = salary * 1.05  
  
where salary < 70000;
```

```
update instructor  
  
set salary = salary * 1.05  
  
where salary < (select avg (salary)
```

from instructor);

update instructor

*set salary = salary * 1.03*

where salary > 100000;

update instructor

*set salary = salary * 1.05*

where salary <= 100000;